
Reproducibility Instructions

Team: Quantum Horizon Africa

Ounimborbitibou Djabon

djabon@aims.edu.gh

Mawulikplimi Roland Hounkpe

rhounkpe@aims.edu.gh

Repository: github.com/mawuliro/wiser_project

Prerequisites. Python 3.10 or later, 4 GB RAM, CPU only. No GPU and no quantum hardware are required; all circuits run on PennyLane’s `default.qubit` state-vector simulator. Every run is checkpointed and resumable.

1. Environment Setup

Clone the repository and install the dependencies. The full stack is pure Python and runs on any operating system.

```
git clone https://github.com/mawuliro/wiser_project.git
cd wiser_project
pip install torch pennylane scipy numpy matplotlib pandas
```

Verified versions: PyTorch 2.0 or later, PennyLane 0.30 or later, SciPy 1.10 or later, NumPy 1.24 or later, Matplotlib 3.7 or later, Pandas 2.0 or later. On Google Colab, the `google-colab` package is also needed for Drive mounting.

2. Phase 1: Classical PINN Baseline

Notebook: `Phase 1/wiser/Phase1_Classical_Baseline.ipynb`

2.1 What the code does

The notebook builds a 9-layer tanh MLP with 3,021 parameters that maps (x, t) to a scalar $u(x, t)$. The network is trained on the 1-D viscous Burgers equation at hard viscosity $\nu = 0.01/\pi$ with initial condition $u(x, 0) = -\sin(\pi x)$ and boundary conditions $u(\pm 1, t) = 0$. The loss is the two-term Raissi form $\mathcal{L} = \mathcal{L}_u + \mathcal{L}_f$, where $\mathcal{L}_u$ is the mean squared error at 100 initial and boundary data points and $\mathcal{L}_f$ is the mean squared PDE residual $f = u_t + u u_x - \nu u_{xx}$ evaluated at 10,000 Latin-Hypercube collocation points. The PDE derivatives are computed exactly by PyTorch autograd.

2.2 Optimizer and iterations

The optimizer is `scipy.optimize.minimize` with `method="L-BFGS-B"` using Raissi’s exact settings: `maxiter=50000`, `maxfun=50000`, `maxcor=50`, `maxls=50`, and `ftol` and `gtol` both set to machine epsilon ($\approx 2.2 \times 10^{-16}$). The model is in float64 throughout. A PyTorch-to-SciPy bridge flattens the model parameters into a single NumPy vector, computes the loss and gradient via PyTorch autograd, and passes them to SciPy. The optimizer ran 50,001 function evaluations and converged to a final loss of 4.97×10^{-8} .

2.3 How to run

```
cd "Phase 1/wiser"
jupyter notebook Phase1_Classical_Baseline.ipynb
```

Run all cells. The notebook writes `results/phase1_burgers_*/metrics.json` with `relative_l2_error` $\approx 2.12 \times 10^{-4}$, three times better than the Raissi et al. target of 6.7×10^{-4} .

3. Phase 2: QAPINN and Classical Twin Grid

Notebook: `Phase 2/phase2_part_1/Tiers12_multi_seed/QAPINN_PHASE2_FINAL.ipynb`

3.1 What the code does

This is the canonical framework source. It defines five model variants sharing one training engine: the Classical PINN, the QAPINN (a VQC replaces the first hidden layer), the parameter-matched Classical Twin (identical parameter count at every width: 1,801 for $n = 3$, 1,827 for $n = 4$, 1,853 for $n = 5$), the GAFF-PINN (every tanh replaced by $\tanh(Naz)$ with one shared trainable scalar a), and the ShahQAPINN (faithful Shah et al. reproduction at 74, 189, 600 parameters).

The QAPINN circuit uses angle encoding with data re-uploading: at each of L layers the input (x, t) is encoded as R_Y rotations on every qubit, followed by trainable R_X rotations and an entanglement block (none, linear, or all-to-all). The measurement is either the n local expectation values $\langle \sigma_z \rangle$ or the full 2^n probability vector. The classical head is a deep tanh MLP of width 20 and depth 5.

The loss is the four-term Shah form $\mathcal{L} = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{bc1}} \mathcal{L}_{\text{bc1}} + \lambda_{\text{bc2}} \mathcal{L}_{\text{bc2}}$ with weights $\lambda_{\text{ic}} = \lambda_{\text{bc1}} = \lambda_{\text{bc2}} = 10$ and $\lambda_{\text{pde}} = 1$. The data-fit terms are up-weighted to prevent the optimizer from collapsing to the trivial zero-output solution.

3.2 Optimizer and iterations

Each configuration is trained in two phases. Phase A is Adam with learning rate 8×10^{-3} and AMSGrad for 1,000 iterations, providing stable escape from initialization. Phase B is PyTorch's L-BFGS with strong-Wolfe line search, history size 60, run in chunks of 200 iterations for 1,000 iterations total, providing high-precision convergence. A blow-up guard restores the best state if the loss diverges by more than $50\times$ the best. The framework checkpoints the full training state every 250 iterations via an atomic `os.replace` write, so interrupted runs resume transparently. A dead-run detector retries with a perturbed seed if the loss stalls across three logged chunks.

The definitive easy-viscosity comparison uses five seeds (1234, 2025, 777, 31415, 8080) and a paired t -test on $\log L^2$. At hard viscosity, three seeds with means and spreads.

3.3 How to run

```
cd "Phase 2/phase2_part_1/Tiers12_multi_seed"
jupyter notebook QAPINN_PHASE2_FINAL.ipynb
```

The notebook skips any configuration that already has a metrics JSON file, so it can be interrupted and re-run at any time. Outputs: `qapinn_runs/results/master_results.csv`, `qapinn_runs/models/*.pt`, `qapinn_runs/phase3.inputs/statistics.json`.

4. Phase 2: Ablation Grid (Optional)

Notebook: Phase 2/phase2_part_2/wiser/QAPINN_ABLATION.ipynb

This notebook runs the 14-configuration ablation grid on easy-viscosity Burgers ($\nu = 0.05$). It varies one design axis at a time with all others fixed: entanglement (none, linear, all-to-all), measurement (expectation values, probability vector), depth ($L = 3$, $L = 9$), and qubit count (3, 4, 5, 6, 7). It also runs a gradient-variance probe for the barren-plateau diagnostic and per-neuron interpretability hooks. The optimizer and schedule are identical to Section 3, except that each configuration uses 1,400 iterations total (1,000 Adam plus 400 L-BFGS).

```
cd "Phase 2/phase2_part_2/wiser"
jupyter notebook QAPINN_ABLATION.ipynb
```

5. Phase 3: Explainability Analysis

Notebook: Phase_3_hardBurger/Phase3_Hard_InputBuilder.ipynb

Phase 3 consumes the Phase 2 artifacts (the `.pt` checkpoints and `.npz` loss histories) and extracts the inputs for three workstreams. No retraining is performed. The InputBuilder rebuilds each trained model once, forward-passes on a 256×256 grid, and computes the metrics at 26 time slices. Workstream A computes the Fourier spectrum of each predicted field and compares it to the analytical Cole-Hopf spectrum. Workstream B measures first-layer gradient variance, loss traces, and wall-clock cost. Workstream C uses Centered Kernel Alignment to trace how much of the final prediction is already present in each layer and measures per-neuron saturation.

```
cd "Phase_3_hardBurger"
jupyter notebook Phase3_Hard_InputBuilder.ipynb
jupyter notebook "figures/Phase3_Hard_Figures.ipynb"
```

The Figures notebook produces HA1 to HA6 (Fourier), HB1 to HB4 (gradient variance and cost), and HC1 to HC8 (CKA and saturation).

6. Hard-Viscosity Grid and Verification

Notebooks: Phase2_hardBurger/ subfolders

The hard-viscosity benchmark extends the width sweep to $n = 3$ through $n = 8$ at $\nu = 0.01/\pi$. Each configuration uses 4,400 iterations (3,000 Adam plus 1,400 L-BFGS). The collocation uses shock-focused Latin Hypercube augmentation: a fraction of points is drawn from a Gaussian centered at $x = 0$ with standard deviation 0.15, concentrated where the shock forms. A time-marching curriculum progressively extends $t_{\max}$.

```
cd "Phase2_hardBurger/QAPINN N3_5" && jupyter notebook N1_QAPINN_hardnu_q345.ipynb
cd "../GAAF-PINN" && jupyter notebook N4_GAAF_hardnu_q3to8.ipynb
cd "../c-PINN" && jupyter notebook N3_TWIN_hardnu_q3to8.ipynb
```

The $n = 8$ runs used a reduced budget of 1,000 iterations and are flagged `in_primary=False` in `run_metadata.csv`.

Verification and troubleshooting. After Phases 1 to 3, confirm: (1) Phase 1 $L^2 = 2.12 \times 10^{-4}$; (2) Phase 2 QAPINN $n=3$ $L^2 = 0.0117 \pm 0.0126$ and $n=5$ $L^2 = 0.00454 \pm 0.00534$ (five-seed means); (3) paired t -test gives $p < 0.05$ only at Burgers $n=5$ (twin wins); (4) Phase 3 Fourier centroid within 1% of analytical for $n \geq 4$; (5) gradient variance in 0.11 to 0.49 across $n = 3$ to 7. If the QAPINN collapses to $L^2 \approx 1.0$, ensure $\lambda_{ic} = \lambda_{bc1} = \lambda_{bc2} = 10$ and $\lambda_{pde} = 1$. If L-BFGS stalls, verify the two-phase schedule is active and checkpoints are written every 250 iterations.